Lab 07 — Resources, Requests, Limits & OOMKilled

ID: K8S-LAB-07 · **Category:** Core · **Level:** Engineer · **Duration:** 45 min

Focus: requests, limits, cgroups, OOMKilled, node allocatable

Objective

Configure resource requests/limits, observe scheduler decisions, and reproduce/recover from OOMKilled.

Concepts Covered

• Difference between requests and limits

• How requests influence scheduling

• How limits become cgroup constraints

Prerequisites

• A cluster from Lab 1

Architecture

Requests are the contract with the scheduler. Limits are the contract with the kernel.

```
requests  ->  scheduler reservation (fit decision)
limits    ->  cgroup hard cap (CPU throttle / OOMKill)
```

Lab Steps

01. Deploy a pod with explicit resources

Asks for 64Mi but tries to use 250Mi → guaranteed OOMKill.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata: {name: capped}
spec:
  containers:
  - name: app
    image: polinux/stress
    resources:
      requests: {memory: "64Mi", cpu: "100m"}
      limits:   {memory: "128Mi", cpu: "200m"}
    command: ["stress","--vm","1","--vm-bytes","250M","--vm-hang","1"]
EOF
```

Expected: STATUS goes Running → OOMKilled within ~10s.

Internals: Kernel OOM killer fires when the memory cgroup is over limit; kubelet reports OOMKilled in the Pod status.

02. Inspect the killed container

Shows Reason=OOMKilled and the exit code (137).

```
kubectl describe pod capped | grep -A2 'Last State'
```

Expected: Reason: OOMKilled, Exit Code: 137.

Internals: Exit code 137 = 128 + SIGKILL(9).

03. Right-size and re-deploy

Pods are mostly immutable; recreate with a higher limit.

```
kubectl patch pod capped --type='json' -p='[{"op":"replace","path":"/spec/containers/0/resources/limits/memor
kubectl delete pod capped && \
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Pod
metadata: {name: capped}
spec:
  containers:
  - name: app
    image: polinux/stress
    resources:
      requests: {memory: "256Mi", cpu: "100m"}
      limits:   {memory: "512Mi", cpu: "200m"}
    command: ["stress","--vm","1","--vm-bytes","250M","--vm-hang","1"]
EOF
```

Expected: Pod stays Running.

Internals: Pod resources are immutable today; in-place resize is alpha behind a feature gate.

04. Inspect node allocatable

Shows what the scheduler thinks is available after system-reserved and kube-reserved.

```
kubectl describe node | grep -A5 Allocatable
```

Expected: CPU/memory values lower than total — that's expected.

Internals: Allocatable = capacity - kubeReserved - systemReserved - evictionThresholds.

Common Errors

OOMKilled in a loop — Cause: Limit too low for the actual working set · Fix: Profile, then raise limit (or fix the leak)

Pod won't schedule — Cause: requests > node allocatable · Fix: Lower requests, or add a node

Troubleshooting

• `kubectl top pod` to see usage if metrics-server is installed

• `dmesg | grep -i oom` on the node confirms kernel OOM

Enterprise Relevance

Mis-set limits cause the majority of mystery production restarts. Every workload spec needs explicit requests and limits.

Validation

• You can reproduce OOMKilled and recover from it

• You can read `Allocatable` and predict whether a Pod will fit

Related Incidents & Guide Sections

Incidents: K8S-INC-3001

• Pod lifecycle & probes

• OOMKilled

Questions

Beginner

Q. What's exit code 137?

A. 128 + SIGKILL(9) — the kernel OOM killer.

Intermediate

Q. Why doesn't setting only limits help the scheduler?

A. The scheduler only looks at requests.

Advanced

Q. What's the difference between system-reserved and kube-reserved?

A. system-reserved is for the OS (sshd etc.); kube-reserved is for kubelet/runtime/cni.

Scenario

Q. A new microservice OOMs every few hours. Approach?

A. Add usage metrics → compute p95 working set → set limit ~1.5x p95, then iterate.

Viva

Q. What QoS class does a Pod get if requests==limits?

A. Guaranteed.